

Lecture 8:

Hamiltonian Monte Carlo

September 21, 2026

in-class: run MCMC using emcee

[ast384c/inclass/lec7_mcmc/Lec7_emcee.ipynb](#)

presentation on MCMC

MCMC is the standard sampling algorithm in Bayesian inference, but what about *very* high dimensional distribution

curse of dimensionality — random walks scale poorly with dimension

MCMC is the standard sampling algorithm in Bayesian inference, but what about *very* high dimensional distribution

curse of dimensionality — random walks scale poorly with dimension

Hamiltonian Monte Carlo (**HMC**) — developed for lattice quantum chromodynamics

lattice QCD gauge configurations have ~60 million dimension!

HMC — efficient sampling of high-dimensional spaces using physics

HMC — efficient sampling of high-dimensional spaces using physics

imagine $-\log p(x)$ as the shape of a frictionless bowl and we're playing a game of marbles

If we roll a marble in this bowl and let it move for a finite time, it'll end up at a different location in the bowl, carried by its momentum

The endpoints of these rolls form a Markov chain that follows the shape of the bowl.

HMC — efficient sampling of high-dimensional spaces using physics

total energy of the marble:

$$E = U + T$$

potential energy kinetic energy

HMC — efficient sampling of high-dimensional spaces using physics

total energy of the marble:

$$E = U + T$$

potential energy kinetic energy

$U(x) = \log p(x)$ from the momentum with
which we push the marble

HMC — efficient sampling of high-dimensional spaces using physics

total energy of the marble:

$$E = U + T$$

potential energy kinetic energy

$U(x) = \log p(x)$ from the momentum with
which we push the marble

trajectory is governed by physics — *i.e.* Hamiltonian Mechanics

HMC — efficient sampling of high-dimensional spaces using physics

$$H(x, y) = U(x) + T(y)$$

x = positions
 y = momenta

HMC — efficient sampling of high-dimensional spaces using physics

$$H(x, y) = U(x) + T(y)$$

x = positions

y = momenta

$$\frac{dx}{dt} = \frac{\partial H}{\partial y}$$

$$\frac{dy}{dt} = -\frac{\partial H}{\partial x}$$

HMC — efficient sampling of high-dimensional spaces using physics

$$H(x, y) = U(x) + T(y)$$

x = positions

y = momenta

$$\frac{dx}{dt} = \frac{\partial H}{\partial y} \qquad \frac{dy}{dt} = - \frac{\partial H}{\partial x}$$

exact due to energy and phase-space conservation (Liouville's theorem) —
if we can integrate exactly

HMC — efficient sampling of high-dimensional spaces using physics

$$H(x, y) = U(x) + T(y)$$

x = positions

y = momenta

$$\frac{dx}{dt} = \frac{\partial H}{\partial y} \qquad \frac{dy}{dt} = - \frac{\partial H}{\partial x}$$

exact due to energy and phase-space conservation (Liouville's theorem) —
if we can integrate exactly *but we can't*

HMC — efficient sampling of high-dimensional spaces using physics

$$H(x, y) = U(x) + T(y)$$

x = positions

y = momenta

$$\frac{dx}{dt} = \frac{\partial H}{\partial y} \qquad \frac{dy}{dt} = - \frac{\partial H}{\partial x}$$

we can solve these using a **symplectic integrator**

HMC — symplectic integrator: *e.g. leapfrog integrator*

$$y(t + \frac{\epsilon}{2}) = y(t) - \left(\frac{\epsilon}{2}\right) \frac{\partial U(x(t))}{\partial x} \quad \begin{array}{l} x = \text{positions} \\ y = \text{momenta} \end{array}$$

$$x(t + \epsilon) = x(t) + \epsilon \frac{\partial T(y(t + \frac{\epsilon}{2}))}{\partial y}$$

$$y(t + \epsilon) = y(t + \frac{\epsilon}{2}) - \left(\frac{\epsilon}{2}\right) \frac{\partial U(x(t + \epsilon))}{\partial x}$$

HMC — symplectic integrator: *e.g. leapfrog integrator*

$$y(t + \frac{\epsilon}{2}) = y(t) - \left(\frac{\epsilon}{2}\right) \frac{\partial U(x(t))}{\partial x} \quad \text{half a momentum kick}$$

$$x(t + \epsilon) = x(t) + \epsilon \frac{\partial T(y(t + \frac{\epsilon}{2}))}{\partial y} \quad \text{full position step}$$

$$y(t + \epsilon) = y(t + \frac{\epsilon}{2}) - \left(\frac{\epsilon}{2}\right) \frac{\partial U(x(t + \epsilon))}{\partial x} \quad \text{other half momentum}$$

HMC — symplectic integrator: *e.g. leapfrog integrator*

$$y(t + \frac{\epsilon}{2}) = y(t) - \left(\frac{\epsilon}{2}\right) \frac{\partial U(x(t))}{\partial x} \quad \text{half a momentum kick}$$

$$x(t + \epsilon) = x(t) + \epsilon \frac{\partial T(y(t + \frac{\epsilon}{2}))}{\partial y} \quad \text{full position step}$$

$$y(t + \epsilon) = y(t + \frac{\epsilon}{2}) - \left(\frac{\epsilon}{2}\right) \frac{\partial U(x(t + \epsilon))}{\partial x} \quad \text{other half momentum}$$

volume is exactly conserved, but *not* energy

more on this in a moment

HMC — we can choose T so that it's simple and analytic

$$y(t + \frac{\epsilon}{2}) = y(t) - \left(\frac{\epsilon}{2}\right) \frac{\partial U(x(t))}{\partial x}$$

$$x(t + \epsilon) = x(t) + \epsilon \frac{\partial T(y(t + \frac{\epsilon}{2}))}{\partial y}$$

$$y(t + \epsilon) = y(t + \frac{\epsilon}{2}) - \left(\frac{\epsilon}{2}\right) \frac{\partial U(x(t + \epsilon))}{\partial x}$$

HMC — but we need **gradients** of $U = \log p(x)$

$$y(t + \frac{\epsilon}{2}) = y(t) - \left(\frac{\epsilon}{2}\right) \frac{\partial U(x(t))}{\partial x}$$

$$x(t + \epsilon) = x(t) + \epsilon \frac{\partial T(y(t + \frac{\epsilon}{2}))}{\partial y}$$

$$y(t + \epsilon) = y(t + \frac{\epsilon}{2}) - \left(\frac{\epsilon}{2}\right) \frac{\partial U(x(t + \epsilon))}{\partial x}$$

HMC — putting it all together

1. initialize x_0
2. draw momentum $y \sim T$
3. run L leapfrog steps of ϵ from (x, y) to (x', y')
4. accept (x', y') with acceptance rate
 $r = \min(1, \exp(H(x, y) - H(x', y')))$; otherwise stay
5. repeat

HMC — putting it all together

1. initialize x_0
2. draw momentum $y \sim T$ *something simple and analytic: e.g. $\mathcal{N}(0, M)$*
3. run L leapfrog steps of ϵ from (x, y) to (x', y')
4. accept (x', y') with acceptance rate
 $r = \min(1, \exp(H(x, y) - H(x', y')))$; otherwise stay
5. repeat

HMC — putting it all together

1. initialize x_0
2. draw momentum $y \sim T$
3. run L leapfrog steps of ϵ from (x, y) to (x', y')
4. accept (x', y') with acceptance rate
 $r = \min(1, \exp(H(x, y) - H(x', y')))$; otherwise stay
5. repeat *correction because energy is not exactly conserved by leapfrog*

in-class: let's implement HMC with leapfrog

[ast384c/inclass/lec8_hmc/hmc_demo.ipynb](#)

aside from the gradient, vanilla HMC also requires tuning ϵ

ϵ too large — H drifts a lot per trajectory and acceptance collapses

ϵ too small — each step barely advances

aside from the gradient, vanilla HMC also requires tuning ϵ

ϵ too large — H drifts a lot per trajectory and acceptance collapses

ϵ too small — each step barely advances

L too small — HMC $\rightarrow$ slow random walk

L too large — trajectory can curve back on itself, wasting gradient evaluations

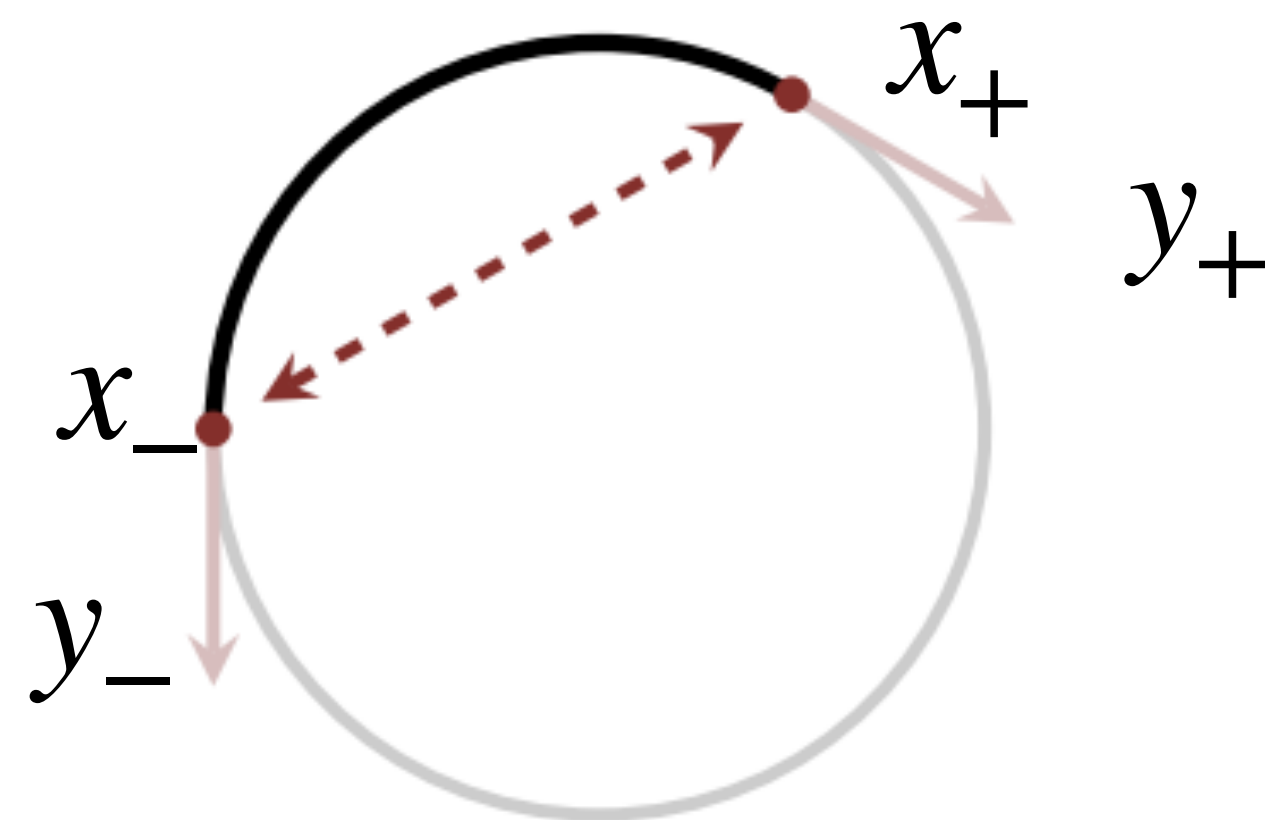
HMC with NUTS — No U-turn Sampler (*Hoffman & Gelman 2014*)

L too small — HMC $\rightarrow$ slow random walk

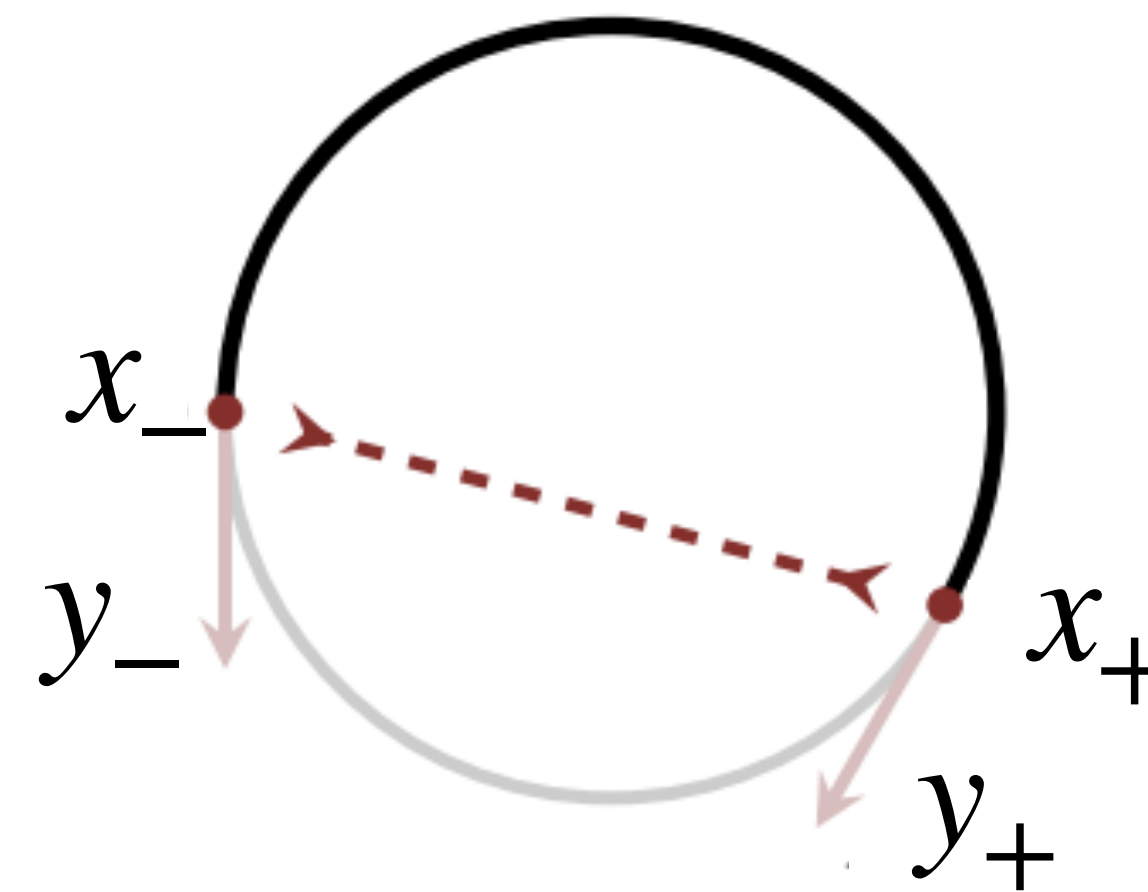
L too large — trajectory can curve back on itself, wasting gradient evaluations, *i.e. a U-turn*

HMC with NUTS — Hamiltonian dynamics is time-reversible

detecting a U-turn: $y_+^T \cdot (x_+ - x_-) < 0$ and
 $y_-^T \cdot (x_- - x_+) < 0$



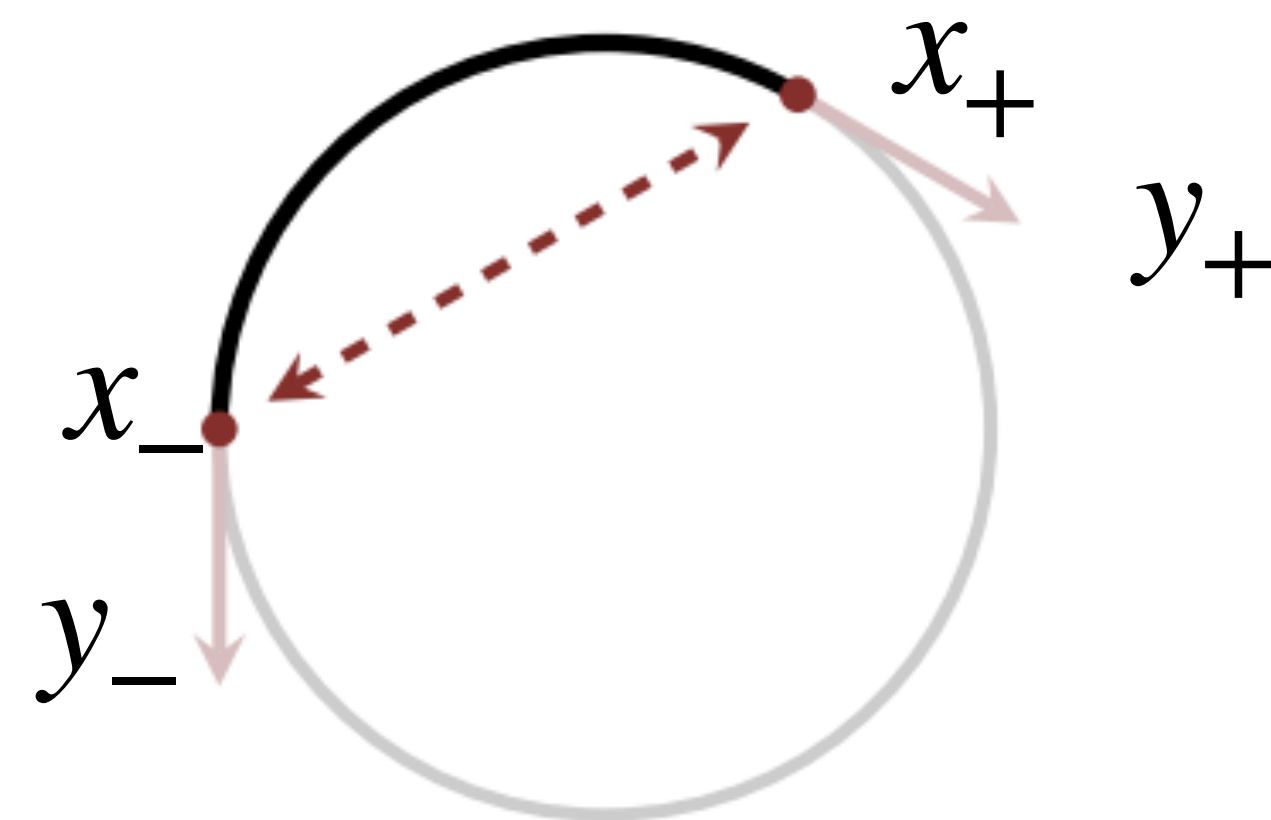
(a)



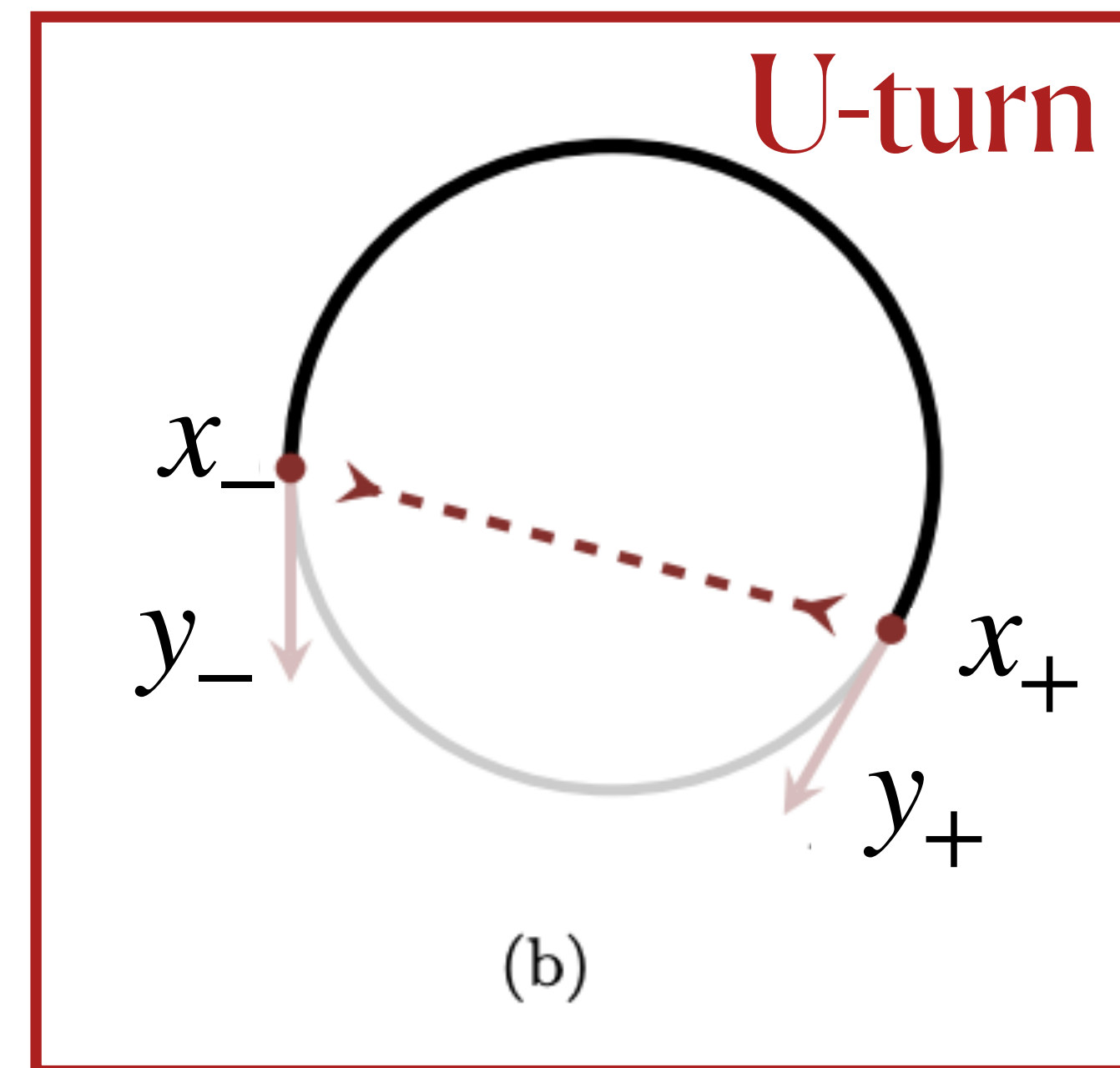
(b)

HMC with NUTS — Hamiltonian dynamics is time-reversible

detecting a U-turn: $y_+^T \cdot (x_+ - x_-) < 0$ and
 $y_-^T \cdot (x_- - x_+) < 0$



(a)



(b)

HMC with NUTS — Hamiltonian dynamics is time-reversible

1. start with $L = 1$
2. each iteration, flip a coin to go forward or backward in time
double L each iteration
3. stop as soon as U-turn criteria is triggered

HMC with NUTS — adaptively tune ε during warm up for a target acceptance rate ~80%

HMC with NUTS — adaptively tune ε during warm up for a target acceptance rate $\sim 80\%$

L set by doubling and no U-turn procedure

HMC with NUTS — adaptively tune ϵ during warm up for a target acceptance rate $\sim 80\%$

L set by doubling and no U-turn procedure

all you need is a differentiable target distribution

HMC with NUTS — adaptively tune ϵ during warm up for a target acceptance rate $\sim 80\%$

L set by doubling and no U-turn procedure

all you need is a differentiable target distribution

automatic differentiation (“autodiff”): JAX, PyTorch, etc

in-class: primer on autodiff using JAX

ast384c/inclass/lec8_hmc/autodiff.ipynb

HMC sampling using NUTS in JAX

ast384c/inclass/lec8_hmc/hmc_nuts.ipynb